Objects

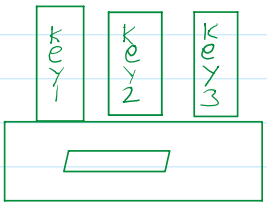
07 September 2026 19:00

As we know from the chapter Data Types, there are eight data types in Javascript. Seven of them are called primitive because their values contain only a single thing (be it a string or a number or whatever).

In contrast, objects are used to store keyed collection of various data and more complex entities. In Javascript, objects penetrate almost every aspect of the language. So we must understand them first before going in depth anywhere else.

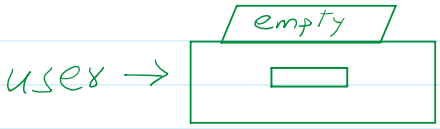
An object can be created with curly braces {…} With an optional list of **properties**. A property is a **"key:value"** pair where **key** is a string (also called a property name) And a **value** can be anything.

We can imagine an object as a cabinet with signed files. Every piece of data is stored in its file by the key. It's easy to find a file by its name or add or remove a file.



An empty object (empty cabinet) can be created using one of the two syntaxes:

```
let user = new Object(); //"object constructor" syntax
        or
let user = {}; //"object literal" syntax
```



Usually, the curly braces {…} are used. That declaration is called an object literal.

Literal and properties

We can immediately put some properties into {... } as *"key:value"* pairs.

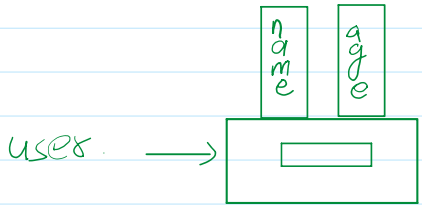
```
let user = {      //an object
    name: "John,  // by key "name" store value "John"
    age: 30       // by key "age" store value 30.
};
```

A property has a key (also known as "name" or "identifier") before the colon ":" and the value to the right of it.

In the user object there are two properties:

- The first property has the name *"name"* and the value *"John"*.
- The second one has the name, *"age"* and the value *30*.

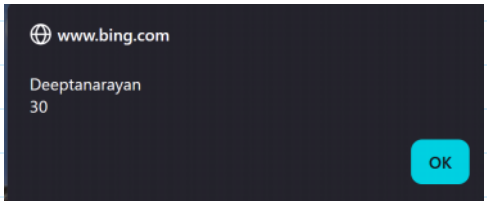
The resulting user object can be imagined as a cabinet with two signed file labeled *name* and *age*.



Property values are accessible using the dot notation.

```
>> let user = {
     name: "Deeptanarayan",
     age: 30
   };
<- undefined
>> user
<- ▶ Object { name: "Deeptanarayan", age: 30 }
```

```
>> alert(user.name)
<- undefined
>> alert(user.age)
<- undefined
>> alert(user.name+"\n"+user.age)
```



The value can be of any type. Let's add a boolean one.

```
>> user.isAdmin = true;
<- true
>> user
<- ▶ Object { name: "Deeptanarayan", age: 30, isAdmin: true }
```

object.name.new property = value

```
>> user.isLoggedIn = "Yes";
<- "Yes"
>> user
<- Object { name: "Deeptanarayan", age: 30, isAdmin: true, isLoggedIn: "Yes" }
```

To remove a property we can use the delete operator:

```
>> delete user.isLoggedIn;
<- true
>> user
<- Object { name: "Deeptanarayan", age: 30, isAdmin: true }
```

The last property in the list may end with a comma.

```
let user = {
   name: "John",
   age: 30,
};
```

That is called a training or hanging, Makes it easier to add, remove, move around properties because all lines become alike.

```
>> let student = {
     roll_num: 101,
       name: `Jones`,
   }
<- undefined
>> student
<- ▶ Object { roll_num: 101, name: "Jones" }
>>
```

Square brackets

For multi world properties the dot access doesn't work:

```
   //user.likes birds = true;
```

Javascript doesn't understand that it thinks that we address user.likes, and then gives us syntax error when comes across unexpected words.

The dot requires the key to be a valid variable identifier. That implies: contains no space. Doesn't start with a digit and doesn't include special characters (dollar '$' and '_' are allowed).

There's an alternative "Square bracket notation" that works with any string:

```
let user = {
    name: "John",
    age: 30,
};
console.log(user);
user[`likes birds`] = true;
console.log(user);
delete user[`likes birds`]
console.log(user)
```

Output:

```
{ name: 'John', age: 30 }
{ name: 'John', age: 30, 'likes birds': true }
{ name: 'John', age: 30 }
```

Now everything is fine. Please note that the string inside the bracket is properly coated. (any type of codes will do).
Square brackets also provide a way to obtain the property name as a result of any expression - as opposed to a literal string - like. From a variable as follows:

```
let user = {
    name: "John",
    age: 30,
};
console.log(user);
let key = `likes birds`;
user[key] = true;
console.log(user)
```

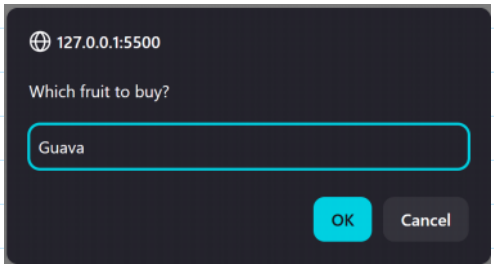
Output:

```
{ name: 'John', age: 30 }
{ name: 'John', age: 30, 'likes birds': true }
```

Here the variable key may be calculated at run - time or depend on the user input. And then we use it to access the property. That gives us a great deal of flexibility.

For instance:

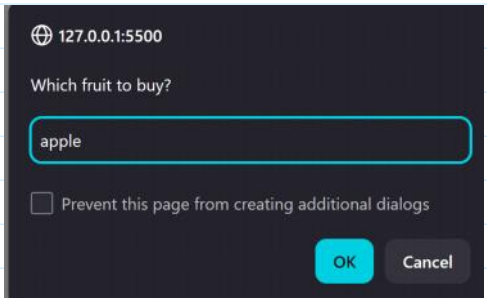
```
let fruit = prompt(`Which fruit to buy? `, `apple`);
let bag= {
    // the name of the property is taken from the variable fruit.
    [fruit]: 5,
};
console.log(bag);
```

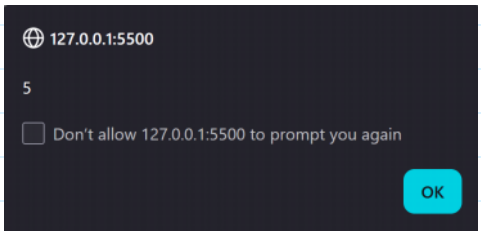



```
Object { Guava: 5 }
```

```
let fruit = prompt(`Which fruit to buy? `, `apple`);
let bag= {
    // the name of the property is taken from the variable fruit.
```

```
    [fruit]: 5,
};
alert( bag.apple );
```





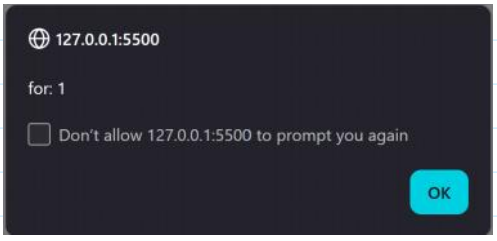
The meaning of a computed property is simple: [fruit] Means that the property name should be taken from fruit.
So if the visitor enters apple bag will become {apple: 5 }.

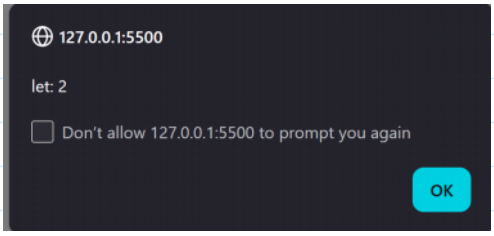
Property names limitations

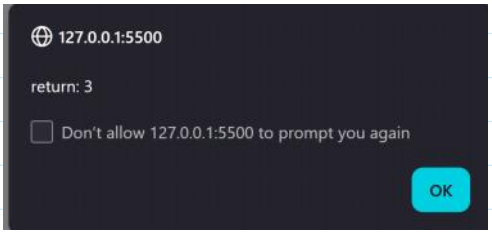
As we already know, a variable cannot have a name equal to one of the language - reserved words like for, let, return, etc.

```
let obj = {
    for: 1,
    let: 2,
    return: 3
};
alert(obj.for)
alert(obj.let)
alert(obj.return)
```

Output:







In short, there are no limitations on property names. They can be any strings or symbols.
Other types are automatically converted to strings.
For instance, a number **0** becomes a string "0" when used as a property key.

```
let obj = {
    for: 1,
    let: 2,
    return: 3,
    0: `test`
};
alert(`for: ${obj.for}`);
alert(`let: ${obj.let}`);
alert(`return: ${obj.return}`);
alert(`0: ${obj[0]}`);
```

Output:
Other output's are above